

# DialGuard

## Intelligent Predictive SmartDialer Prototype for Collections Operations

Deterministic Safety Controls • High-Concurrency Atomic Allocation • Fault-Tolerant Telecom Lifecycle

**Core Engineering Principle:**

"Optimization is strictly subordinate to safety. While predictive pacing statistically forecasts call completions to maximize human agent utilization, the deterministic Safety Controller retains absolute non-bypassable authority to throttle or halt dials whenever capacity or carrier health degrades."

|                   |                                                                          |                    |                                                                                                 |
|-------------------|--------------------------------------------------------------------------|--------------------|-------------------------------------------------------------------------------------------------|
| Project:          | DialGuard Prototype                                                      | Author / Engineer: | Ayush Patil                                                                                     |
| Language / Stack: | Python 3.12 (Standard Library)                                           | Concurrency Model: | Thread-Safe In-Memory Store (RLock)                                                             |
| Test Suite:       | 96 / 96 Pytest Unit & Load Tests Passing                                 | Repository:        | <a href="https://github.com/ayush300302/DialGuard">https://github.com/ayush300302/DialGuard</a> |
| Core Modules:     | State Machines, Allocator, Telecom, Safety, Pacing, Recovery, Simulation | Date:              | August 2026                                                                                     |

**Report Contents:** 1. Problem Statement • 2. System Architecture • 3. Domain State Machines • 4. End-to-End Call Lifecycle • 5. Progressive vs Predictive Dialing • 6. Deterministic Safety Controller • 7. Atomic Concurrency & Allocation • 8. Telecom Carrier Faults & Idempotency • 9. Failure Recovery Supervisor • 10. Simulation Benchmarks • 11. Verification & Test Suite • 12. Trade-offs & Roadmap

## 1. PROBLEM STATEMENT & DOMAIN CONTEXT

Collections contact operations handle high volumes of outbound phone interactions to recover debt. The operation relies on two critical entities:

- **Borrowers:** The individuals being contacted regarding outstanding accounts.
- **Collections Agents:** Human employees who converse with borrowers once a live call is established. This system does *not* utilize AI/LLM conversational agents; human labor is the primary high-cost bottleneck.

**The Operational Trade-off:**

1. *Under-Dialing (Manual / Slow Progressive):* Outbound dials are placed conservatively. With typical borrower answer rates often hovering between 15% and 35%, human agents spend up to 70–85% of their working hours idling in silence waiting for a connection.
2. *Over-Dialing (Unconstrained Predictive):* The dialer aggressively places numerous calls anticipating high drop-offs. If multiple borrowers pick up simultaneously, answered calls exceed available agents, causing **abandoned calls** (silent drops), severe regulatory penalties (e.g. TCPA/FTC 3% maximum abandon rate limits), and hostile borrower experiences.

**DialGuard Objective:** Build a deterministic SmartDialer prototype that dynamically maximizes agent utilization via statistical predictive pacing while guaranteeing deterministic, non-bypassable safety controls and complete fault tolerance.

## 2. SYSTEM ARCHITECTURE & PIPELINE FLOW

DialGuard enforces a strict, layered uni-directional pipeline. The Predictive Pacing Engine produces advisory recommendations, but is structurally decoupled from call dispatching. All requests must pass through the Safety Controller.

| Pipeline Stage       | Component                | Core Responsibility & Operational Role                                                                                       |
|----------------------|--------------------------|------------------------------------------------------------------------------------------------------------------------------|
| 1. Input Metrics     | Historical State Store   | Tracks active agents, in-flight ringing calls, historical answer rates, and average call duration.                           |
| 2. Advisory Pacing   | Predictive Pacing Engine | Statistically estimates target dial count using answer rates and Little's Law completion lookahead. <i>Advisory only.</i>    |
| 3. Safety Gatekeeper | Safety Controller        | <b>Non-bypassable arbiter.</b> Enforces hard capacity caps (available agents, max overdial ratio, carrier health fallbacks). |
| 4. Atomic Allocation | Call Allocator           | Atomically queries and reserves available agents and queued calls with time-bounded leases under RLock.                      |
| 5. Carrier Dispatch  | Telecom Provider         | Dispatches outbound calls to carriers (simulated via ReliableProvider and FlakyProvider).                                    |
| 6. Event Ingestion   | Provider Event Handler   | Idempotently processes carrier signals (INITIATED, RINGING, ANSWERED, COMPLETED) with deduplication.                         |
| 7. Fault Recovery    | Recovery Supervisor      | Asynchronously sweeps expired worker leases and stuck in-flight calls, returning orphaned records to clean pools.            |

### 3. DOMAIN STATE MACHINES

State transitions are strictly centralized, immutable, and deterministic. Invalid transitions immediately raise domain exceptions (`InvalidStateTransitionError` or `TerminalStateError`) and guarantee that entity state remains untouched.

#### A. Collections Agent State Machine

| Agent State | Permitted Next States                  | Business & Operational Rationale                                                      |
|-------------|----------------------------------------|---------------------------------------------------------------------------------------|
| OFFLINE     | AVAILABLE, PAUSED                      | Agent logs in ready to accept calls or logs in directly on break/meeting.             |
| AVAILABLE   | RESERVED, PAUSED, OFFLINE              | Agent is idle in queue; can be reserved by allocator, take a break, or log off.       |
| RESERVED    | DIALING, AVAILABLE, OFFLINE            | Locked for outbound dial. Moves to DIALING on carrier dial, or AVAILABLE on timeout.  |
| DIALING     | CONNECTED, WRAP_UP, AVAILABLE, OFFLINE | Call placed. Moves to CONNECTED on pickup, WRAP_UP/AVAILABLE on no-answer/busy.       |
| CONNECTED   | WRAP_UP, OFFLINE                       | Active conversation with borrower. Concludes into post-call disposition wrap-up.      |
| WRAP_UP     | AVAILABLE, PAUSED, OFFLINE             | Agent logging notes/disposition. Returns to active queue, takes a break, or logs out. |
| PAUSED      | AVAILABLE, OFFLINE                     | Agent on break or meeting. Resumes availability or logs off.                          |

#### B. Call Lifecycle State Machine

| Call State | Permitted Next States                | Terminal?  | Carrier & Lifecycle Meaning                                              |
|------------|--------------------------------------|------------|--------------------------------------------------------------------------|
| QUEUED     | RESERVED, CANCELLED                  | No         | Scheduled for dialing; reserved by allocator or cancelled (e.g. DNC).    |
| RESERVED   | INITIATED, QUEUED, CANCELLED         | No         | Locked for agent. Moves to INITIATED on dial, QUEUED on lease expiry.    |
| INITIATED  | RINGING, ANSWERED, FAILED, CANCELLED | No         | Dial request dispatched to carrier. Awaits network ringing or failure.   |
| RINGING    | ANSWERED, FAILED, CANCELLED          | No         | Handset ringing. Advances to ANSWERED on pickup, FAILED on busy/timeout. |
| ANSWERED   | CONNECTED, COMPLETED, FAILED         | No         | Borrower answered. Bridges to agent (CONNECTED) or IVR (COMPLETED).      |
| CONNECTED  | COMPLETED, FAILED                    | No         | Active conversation. Terminates normally (COMPLETED) or drops (FAILED).  |
| COMPLETED  | <i>None (Terminal)</i>               | <b>YES</b> | Successful call conclusion. Locked against all further transitions.      |
| FAILED     | <i>None (Terminal)</i>               | <b>YES</b> | Unanswered, busy, invalid number, carrier timeout, or network drop.      |
| CANCELLED  | <i>None (Terminal)</i>               | <b>YES</b> | Aborted prior to connection (campaign stop, DNC match, initiation drop). |

## 4. END-TO-END CALL FLOW TRACE

The following concrete lifecycle trace illustrates how a single call (C1) and borrower (B1) interact with human Collections Agent (A1) across all system boundaries:

| Step & Event        | Agent State | Call State | Governing Component & Action Description                                                          |
|---------------------|-------------|------------|---------------------------------------------------------------------------------------------------|
| 1. Initial State    | AVAILABLE   | QUEUED     | Agent is idle in queue; Call C1 is scheduled in repository.                                       |
| 2. Allocation       | RESERVED    | RESERVED   | <b>Call Allocator:</b> Atomically reserves A1 and C1 under lock; assigns 30s lease timestamp.     |
| 3. Provider Dial    | DIALING     | INITIATED  | <b>Telecom Provider:</b> Dispatches carrier request; emits INITIATED event.                       |
| 4. Carrier Ringing  | DIALING     | RINGING    | <b>Provider Event Handler:</b> Receives RINGING event; transitions C1 state.                      |
| 5. Borrower Pickup  | CONNECTED   | CONNECTED  | <b>Provider Event Handler:</b> Receives ANSWERED, immediately bridges call to Agent A1.           |
| 6. Call Termination | WRAP_UP     | COMPLETED  | <b>Carrier &amp; Handler:</b> Call ends. C1 enters terminal COMPLETED; A1 enters post-call notes. |
| 7. Notes Finished   | AVAILABLE   | COMPLETED  | <b>Agent UI / Workflow:</b> Agent completes disposition notes and returns to active queue.        |

## 5. PROGRESSIVE VS. PREDICTIVE DIALING

### Progressive Dialing (1:1 Ratio):

Strict rule: available agents = maximum number of agent-bound outbound calls allowed at that moment.

If 10 agents are available, exactly 10 outbound calls are placed. This mode guarantees zero abandoned calls, but in low answer-rate environments (e.g. 20%), human agents remain idle waiting for lines to connect.

### Predictive Pacing Engine (Statistical Forecasting):

Estimates required dial volume based on real-time empirical signals without deploying heavyweight machine learning models:

#### Pacing Formula:

**Target Answers** =  $\max(0, (\text{Available Agents} + \text{Expected Wrap-ups}) - (\text{In-flight Calls} \times \text{Answer Rate}))$

**Recommended Dials** =  $\text{Target Answers} / \text{Effective Answer Rate}$

**Little's Law Lookahead:**  $\text{Expected Wrap-ups} = (\text{Connected Calls} / \text{Avg Talk Time}) \times \text{Dial Latency Window (5.0s)}$

**Crucial Design Distinction:** The Predictive Pacing Engine produces *recommendations only*. It has zero authority to dispatch calls or allocate resources directly. Its recommendation is passed to the Safety Controller for validation.

## 6. DETERMINISTIC SAFETY CONTROLLER

The Safety Controller is the non-bypassable guardian of the dialing system. It evaluates operational state and returns a binding decision:

- Zero Available Capacity:** If 0 agents are AVAILABLE, all dials are strictly rejected (approved = 0).
- Overdial Ceiling:** Total in-flight dials + new dials cannot exceed  $\text{available\_agents} \times \text{max\_overdial\_ratio}$  (default: 3.0).
- Abandonment Risk Cap:** Expected answered calls are constrained so they never exceed available agent capacity.
- Carrier Health Degradation Fallback:** If carrier health drops below 0.70, predictive dialing is automatically throttled and forced back to 1:1 progressive dialing.
- Critical Carrier Failure:** If carrier health falls below 0.30, all dials are halted completely.

*Note: Numerical thresholds (3.0 overdial ratio, 0.70 degradation fallback, 0.30 critical cutoff) are configurable implementation parameters, not hardcoded assignment mandates.*

## 7. CONCURRENCY & ATOMIC ALLOCATION

In a multi-worker contact center, multiple background worker processes concurrently attempt to match available agents with queued calls. Without concurrency controls, race conditions lead to **double-booking agents** (two calls connecting to the same agent) or **double-dialing borrowers**.

| Concurrent Contention                                                                                                                                              |                                                                                                                                                                                                                            | Deterministic Resolution                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Worker 1 (Thread A)</li><li>• Worker 2 (Thread B)</li></ul> <p><i>Both attempt to allocate Agent A1 simultaneously</i></p> | <p>[ RLock Critical Section ]</p> <ol style="list-style-type: none"><li>1. Atomically verify Agent A1 AVAILABLE</li><li>2. Atomically verify Call C1 QUEUED</li><li>3. Verify Borrower has no other active calls</li></ol> | <ul style="list-style-type: none"><li>• Worker 1: Acquires lock, reserves A1 &amp; C1 (SUCCESS)</li><li>• Worker 2: Sees A1 already RESERVED, gracefully skips (SAFE)</li></ul> |

**Concurrency Implementation (Prototype Design):**

- Uses threading.RLock protecting all read-modify-write operations inside InMemoryRepository.
- reserve\_agent\_and\_call() atomically checks that the agent is AVAILABLE, call is QUEUED, and borrower has no other active calls before simultaneously transitioning both entities to RESERVED.
- **Stress Test Validation:** 30 concurrent worker threads attempting rapid allocations across 500 calls and 50 agents achieved **0 double-bookings** and **0 duplicate borrower allocations** (verified in test\_load\_concurrency.py).

## 8. TELECOM PROVIDER FAULTS & IDEMPOTENCY

Real-world telecom carriers frequently exhibit erratic behaviors. DialGuard models two carrier implementations to validate resilience:

- **Provider 1 (ReliableProvider):** Fast, deterministic, orderly event flow.
  - **Provider 2 (FlakyProvider):** Injects realistic anomalies including network timeouts, duplicate transmissions, inverted out-of-order deliveries (e.g. ANSWERED before RINGING), and dynamic health score degradation.

**Idempotency & Resilience Mechanisms:**

1. *Event ID Deduplication:* Every incoming event has a unique event\_id cached in memory. Duplicate events are silently ignored.
2. *Milestone Deduplication:* (call\_id, event\_type) milestones prevent re-executing identical state transitions.
3. *Terminal State Lockdown:* Calls in COMPLETED, FAILED, or CANCELLED strictly reject subsequent late-arriving carrier events, preventing state corruption.
4. *Direct Answer Progression:* If carrier skips ringing and reports ANSWERED directly from INITIATED, the state machine advances smoothly without error.

## 9. FAILURE RECOVERY SUPERVISOR

Worker state is never the sole source of truth. If a worker process crashes after reserving an agent and call, the RecoverySupervisor automatically recovers orphaned resources:

- **Lease Expiration Sweeps:** Each reservation attaches a 30s lease timestamp (lease\_expires\_at). The supervisor periodically sweeps the repository, returning expired RESERVED calls to QUEUED and orphaned agents to AVAILABLE.
- **Stuck In-Flight Recovery:** Calls stuck in INITIATED or RINGING beyond the timeout threshold (60s) without carrier updates are transitioned to FAILED, freeing the assigned agent back to AVAILABLE.

## 10. SIMULATION RESULTS & BENCHMARKS

DialGuard includes a discrete-event campaign simulator benchmarking **Progressive vs. Predictive Dialing** across the four standard scenarios. Below are the actual results from the latest execution:

| Scenario & Context                                                       | Mode        | Dials | Answered | Completed | Failed / Unans. | Agent Util.  | Safety Caps | Fallback Cycles |
|--------------------------------------------------------------------------|-------------|-------|----------|-----------|-----------------|--------------|-------------|-----------------|
| <b>Scenario A</b><br>Low Answer: 20%<br>Avg Talk: 120s                   | Progressive | 200   | 43       | 43        | 157             | 40.8%        | 0           | N/A             |
|                                                                          | Predictive  | 200   | 42       | 42        | 158             | <b>44.3%</b> | 60          | 0               |
| <b>Scenario B</b><br>Moderate Answer: 50%<br>Avg Talk: 90s               | Progressive | 200   | 101      | 101       | 99              | <b>55.5%</b> | 0           | N/A             |
|                                                                          | Predictive  | 200   | 90       | 90        | 110             | 52.7%        | 51          | 0               |
| <b>Scenario C</b><br>High Answer: 70%<br>Avg Talk: 180s                  | Progressive | 103   | 72       | 66        | 31              | <b>82.8%</b> | 0           | N/A             |
|                                                                          | Predictive  | 103   | 74       | 66        | 29              | <b>82.8%</b> | 59          | 0               |
| <b>Scenario D</b><br>Dynamic Shift: 40%→15%<br>Flaky Carrier Degradation | Progressive | 173   | 58       | 58        | 115             | 44.5%        | 0           | N/A             |
|                                                                          | Predictive  | 169   | 64       | 64        | 105             | 41.2%        | 60          | <b>59</b>       |

### Technical Interpretation:

- *Stochastic Realities*: Predictive pacing is not guaranteed to outperform progressive dialing in every random distribution. In Scenario B, progressive achieved slightly higher utilization due to random cluster pickups.
- *Low-Answer Regimes (Scenario A)*: Predictive pacing improved agent utilization (+3.5%) by forecasting completions and buffering in-flight dials.
- *Dynamic Carrier Degradation (Scenario D)*: When carrier health degraded, the Safety Controller actively intervened, executing **59 progressive fallback cycles** to shield human agents from carrier dropouts.

## 11. VERIFICATION & TEST SUITE COVERAGE

**100% Passing Test Suite:** The test suite consists of **96 automated tests** executed via Pytest in **0.19 seconds**:

| Test Module & File        | Tests | Key Behaviors & Invariants Covered                                                                                |
|---------------------------|-------|-------------------------------------------------------------------------------------------------------------------|
| test_agent_state.py       | 35    | All 19 valid transitions, illegal jumps, self-transitions, state immutability on failure, full shift lifecycles.  |
| test_call_state.py        | 29    | Happy path, direct answer, terminal state lockdown (COMPLETED/FAILED/CANCELLED), out-of-order rejection.          |
| test_allocator.py         | 5     | Atomic reservations, multi-worker agent double-booking prevention, duplicate borrower call rejection.             |
| test_telecom.py           | 7     | ReliableProvider event flow, FlakyProvider timeout, duplicate, and out-of-order injection.                        |
| test_event_handler.py     | 5     | Event deduplication cache, milestone filtering, terminal call protection, agent-call synchronization.             |
| test_safety_controller.py | 6     | Hard capacity limits, overdial ratio caps, zero agent rejection, critical cutoff, progressive fallback.           |
| test_pacing_engine.py     | 4     | Pacing formulas, Little's Law wrap-up lookahead, in-flight deductions, carrier health scaling.                    |
| test_dialers.py           | 2     | Progressive 1:1 agent limit enforcement, Predictive dialer pipeline orchestration.                                |
| test_recovery.py          | 2     | Worker crash simulation, reservation lease expiration sweeps, stuck in-flight call timeouts.                      |
| test_load_concurrency.py  | 1     | <b>High-concurrency stress test:</b> 30 threads, 500 calls, 50 agents -> 0 double allocations, intact invariants. |

## 12. DESIGN DECISIONS & ARCHITECTURAL TRADE-OFFS

| Architectural Decision     | Implementation Choice                   | Trade-off & Engineering Rationale                                                                                       |
|----------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| In-Memory Store vs Real DB | InMemoryRepository with threading.RLock | Zero external dependencies (no Postgres/Redis); ultra-fast execution. Trade-off: volatile state on process termination. |
| Advisory Pacing Engine     | Pacing recommends; Safety decides       | Separates statistical heuristic optimization from safety invariant enforcement. Pacing can never bypass safety.         |
| Deterministic Safety Gate  | Hard mathematical capacity checks       | Deterministic behavior is auditable and explainable in technical reviews, avoiding opaque ML hallucinations.            |
| Lease-Based Crash Recovery | Time-bounded reservation leases (30s)   | Prevents orphaned locks when workers crash without requiring heavyweight distributed consensus.                         |
| Telecom Provider Mocks     | ReliableProvider & FlakyProvider        | Allows deterministic simulation of network chaos (timeouts, duplicates, out-of-order events) without telephony costs.   |

## 13. LIMITATIONS & PRODUCTION ROADMAP

DialGuard was intentionally constructed as a self-contained local prototype. To transition from prototype to an enterprise-grade telecom platform, the following enhancements represent the production roadmap (*none of which are currently claimed as implemented*):

- **Distributed Persistence & Storage:** Transitioning from in-memory dictionaries to PostgreSQL with row-level locks (FOR UPDATE SKIP LOCKED) or Redis distributed locks (Redlock).
- **Real Telephony Gateway:** Integrating SIP trunking, Webhooks, or Asterisk/FreeSWITCH / Twilio Voice APIs with Answering Machine Detection (AMD).
- **Regulatory Compliance Engine (TCPA / TSR):** Hard enforcement of the FTC 3% maximum call abandonment safe harbor rule and geographic calling time windows (8:00 AM – 9:00 PM local borrower time).
- **Streaming Event Bus:** Decoupling provider event handling via Kafka or RabbitMQ event streams for horizontal worker scalability.
- **Bayesian Pacing Models:** Upgrading statistical pacing with online Bayesian updates for dynamic multi-campaign answer rate forecasting.

## 14. CONCLUSION & KEY ENGINEERING TAKEAWAYS

**Summary Axiom: "Optimization is Subordinate to Safety"**

*DialGuard demonstrates that high collections-agent productivity does not require risky, unconstrained dialing or opaque machine learning models. By enforcing deterministic domain state machines, atomic reservation locks, and a non-bypassable Safety Controller, the system maintains ironclad operational safety while dynamically optimizing outbound call throughput.*

DialGuard Prototype • Repository: <https://github.com/ayush300302/DialGuard> • Author: Ayush Patil